Locating and Traversing Records

In the Depot application, `LineItems` have direct relationships to three other models: `Cart`, `Order`, and `Product`. Additionally, models can have indirect relationships mediated by resource objects. The relationship between `Orders` and `Products` through `LineItems` is an example of such a relationship.

All of this is made possible through IDs.

Identifying Individual Rows

Active Record classes correspond to tables in a database. Instances of a class correspond to the individual rows in a database table. Calling `Order.find(1)`, for instance, returns an instance of an `Order` class containing the data in the row with the primary key of `1`.

If you're creating a new schema for a Rails application, you'll probably want to go with the flow and let it add the `id` primary key column to all your tables. But if you need to work with an existing schema, Active Record gives you a way of overriding the default name of the primary key for a table.

For example, we may be working with an existing legacy schema that uses the ISBN as the primary key for the `books` table.

We specify this in our Active Record model using something like the following:

```
class LegacyBook < ApplicationRecord
  self.primary_key = "isbn"
end
```

Normally, Active Record takes care of creating new primary key values for records that we create and add to the database—they'll be ascending integers (possibly with some gaps in the sequence). However, if we override the primary key column's name, we also take on the responsibility of setting the primary key to a unique value before we save a new row. Perhaps surprisingly, we still set an attribute called `id` to do this. As far as Active Record is concerned, the primary